Security Test Report

FalconStrix – SQL Injection Assessment Across Multiple Input Vectors

Report ID	FALSTRX-2026-006
Components Tested	Login form; alert message field; alert-ID parameter; username-availability endpoint
Vulnerability Class Tested	SQL Injection – CWE-89
Result	No vulnerability found across 4 tested input vectors
Discovered By	Manual black-box testing (Burp Suite), payload-based probing
Environment	Isolated local lab – Windows 11 target, Kali Linux (VMware) attacker
Assessment Date	26 July 2026
Status	Closed – No Action Required

1. Summary

This assessment tested FalconStrix for SQL Injection (SQLi) across four distinct user-controllable input points: the login form, the manual alert creation message field, the numeric alert-ID URL parameter used in alert resolution, and the `username` query parameter on the signup page's availability-check endpoint. Each vector was tested with classic error-inducing and logic-altering SQLi payloads. All four vectors were found to correctly and independently resist injection, through three different underlying defensive mechanisms: parameterized database queries, strict Flask URL-routing type conversion, and server-side input allow-list validation. No exploitable SQL injection was identified in this assessment.

2. Methodology

For each candidate input, testing followed a consistent progression: first a simple logic-altering payload (e.g. `' OR '1'='1`) to check whether application behavior changed unexpectedly; then a minimal, single-character payload (a lone `'`) to check for SQL syntax errors, which are often the clearest signal of unescaped input reaching a live query. Server responses, status codes, and (where relevant, since `debug=True` was enabled on the target) any error detail or traceback content were inspected for each attempt.

3. Vector 1 – Login Form

Payload `administrator'--` submitted as the username field, intended to comment out the remainder of a hypothetical unparameterized query and bypass password verification.

```
POST /login HTTP/1.1
username=administrator%27--&password=wrong123

--- Response ---
HTTP/1.1 200 OK (re-rendered login page, not a 302 redirect to /dashboard)
```

```
"Invalid credentials."
```

Result: Login was not bypassed; the payload was treated as a literal (invalid) username. Consistent with parameterized query usage confirmed in prior source review of `auth_service.py`.

4. Vector 2 – Manual Alert Message Field

4.1 Logic-based payload

```
POST /api/alerts/manual HTTP/1.1
{"severity":"MEDIUM","message":"testing' OR '1'='1"}

--- Response ---
HTTP/1.1 200 OK
{"alert_id":250, "ok":true, "snapshot": {"active_alerts": [
{"message": "testing' OR '1'='1", ...}
]}}
```

4.2 Lone single-quote payload

```
POST /api/alerts/manual HTTP/1.1
{"severity":"MEDIUM","message":"'"}

--- Response ---
HTTP/1.1 200 OK
{"alert_id":251, "ok":true, "snapshot": {"active_alerts": [
{"message": "'", ...}
]}}
```

Result: Both payloads, including an unclosed single quote that would break an unparameterized SQL string, were stored and returned as exact literal text with no error, no broken query, and no unexpected side effects. This is the expected signature of a parameterized `INSERT` statement, where user input is bound as a value and never merged into the SQL command string itself.

5. Vector 3 – Alert-ID URL Parameter

5.1 Non-existent numeric ID (control test)

```
POST /api/alerts/99999/resolve HTTP/1.1

--- Response ---
HTTP/1.1 404 NOT FOUND
{"ok": false, ...} (application-level JSON 404)
```

5.2 Single-quote in place of the ID

```
POST /api/alerts/'/resolve HTTP/1.1

--- Response ---
HTTP/1.1 404 NOT FOUND
<!doctype html>...Werkzeug's generic HTML 404 page...
```

Result: The two 404 responses differ in origin, which is itself informative: the numeric-but-nonexistent ID reached the application's own route handler and its JSON 404 response; the non-numeric quote character was rejected at Flask's URL-routing layer itself (consistent with an `<int:alert_id>` route converter), before the request ever reached application code or a database query. The ID parameter cannot carry SQL injection payloads because non-numeric input never passes routing.

6. Vector 4 – Username-Availability Query Parameter

6.1 Baseline request

```
GET /api/auth/username-availability?username=zaid HTTP/1.1

--- Response ---
HTTP/1.1 200 OK
{"available": false, "message": "Username is already taken.", "ok": true}
```

6.2 Logic-based payload (properly URL-encoded)

```
GET /api/auth/username-availability?username=zaid%27%20OR%20%271%27%3D%271 HTTP/1.1
(decodes to: username=zaid' OR '1'='1)

--- Response ---
HTTP/1.1 200 OK
{"available": false,
"message": "Only letters, numbers, underscore, hyphen, or dot allowed.",
"ok": true}
```

Result: The endpoint rejected the payload outright via server-side allow-list validation on the character set, before any database lookup could be influenced by SQL-meaningful characters (quotes, spaces used as operators, etc.). This is a distinct, input-validation-layer defense, separate from parameterization.

A note on methodology: an initial attempt against this endpoint using an unencoded payload and an incorrect parameter name (`u` instead of `username`) produced a `400 Bad request syntax` error caused by literal, unencoded spaces breaking the HTTP request line itself — this was a test-construction error, not a finding, and was corrected before reaching the result above.

7. Summary of Defensive Mechanisms Observed

Vector	Payload Type	Result	Defense Mechanism
Login form	Auth-bypass (' --)	No bypass	Parameterized query
Alert message field	Logic + lone quote	Stored as literal text	Parameterized query
Alert-ID URL parameter	Lone quote	404 at routing layer	Flask <int:...> type conversion
Username availability param	Logic (encoded)	Rejected outright	Server-side character allow-list

8. Analysis & Conclusion

SQL Injection requires user-supplied data to be interpreted as part of a SQL command rather than as inert data. Across all four tested vectors, FalconStrix consistently prevented this through independent, layered mechanisms rather than a single point of protection: database queries bind user input as parameters rather than concatenating it into SQL strings, the web framework's own routing layer rejects malformed types before they reach application

code, and at least one endpoint additionally enforces a strict input character allow-list. This layered consistency, observed across unrelated parts of the codebase, suggests SQL injection resistance here is a structural property of how the application was built, rather than an accident of one well-written function.

Conclusion: No SQL Injection vulnerability identified in any of the four vectors tested in this assessment.

9. Value of This Test

As with the earlier IDOR assessment on this application, a well-documented negative result demonstrates that a vulnerability class was actively and thoroughly tested — across multiple, structurally different input types — rather than left unexamined. This distinguishes a genuine security assessment from a superficial scan, and the specific payload/response evidence recorded here would allow the same tests to be efficiently re-run if the underlying code changes in the future.

10. Testing Environment

- Target: FalconStrix on Windows 11, Flask bound to 0.0.0.0:5001, isolated VMware network, debug mode enabled (increasing the likelihood that any real injection would have surfaced visible errors/tracebacks during testing).
- Attacker: Kali Linux (VMware), Burp Suite Community Edition (Repeater used for all payload construction and replay).
- All testing performed exclusively against the researcher's own locally-hosted project.
